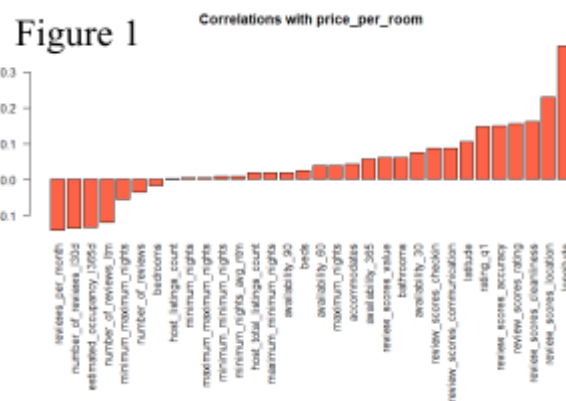


ACTL3142 Assignment 1

Simon Teng z5690607

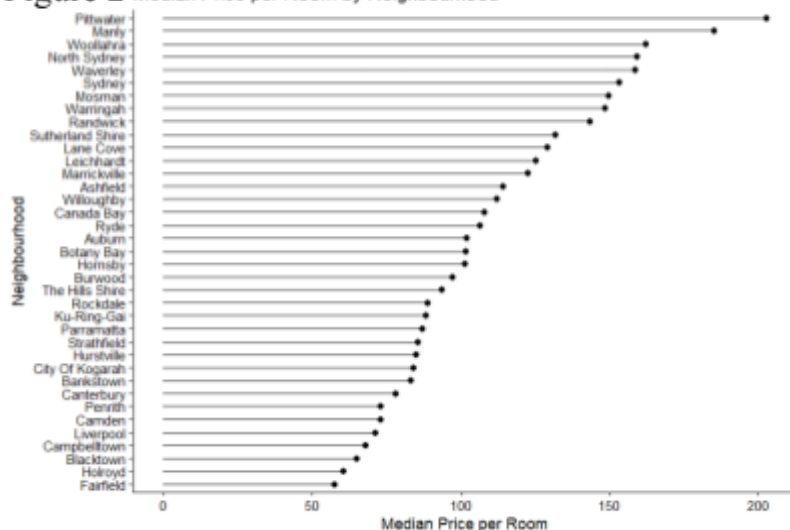
1. An initial exploration of the data was calculated by calculating correlations between numerical predictors and price per room. This yielded the results presented in Figure 1. These correlations provide a general indication of predictor strength, notably revealing longitude and reviews per month as strong candidate predictors. Additionally, it can be seen that the review_score predictors have relatively high and similar correlations to price per room of around 0.15.



The relationship between review scores over various categories and the price_per_room is a reflection of the quality of the listing and is demonstrated by the positive correlations. Higher quality listings naturally demand a premium and will attract more interest. Moreover, due to the high correlation between different scores, caution should be taken from the usage of these predictors as multicollinearity may be introduced from several of these variables capturing similar underlying effects.

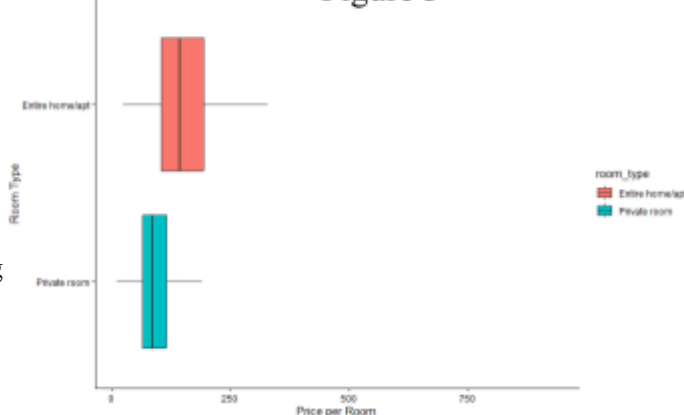
Less intuitive is the relatively strong negative correlation of -0.14 between review frequency and the prices per room. Lower-priced listings attract a broader customer base resulting in a greater number of bookings and subsequent reviews left on the listings. Thus, this predictor has an indirect but meaningful effect on the price per room.

Figure 2 Median Price per Room by Neighbourhood



Another categorical predictor that would meaningfully affect the price per room would be the predictor room_type with two categories of “Entire home/apt” and “Private room.” Intuitively, listings advertised as home/apts will be charged a premium due to the added luxury provided from exclusive access to a location and the opportunity costs for hosts who lose the option to provide multiple private rooms. This intuition is reflected in Figure 3, demonstrating private rooms to have a lower median price of \$86 compared with \$146 for entire homes/apts.

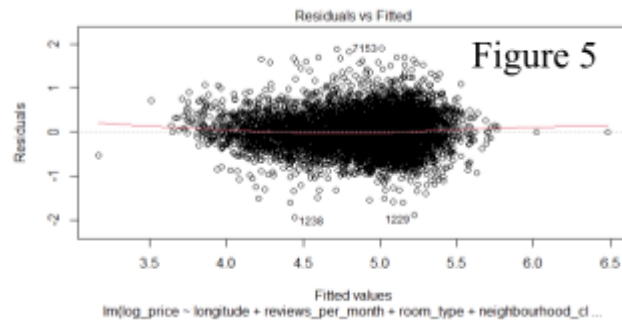
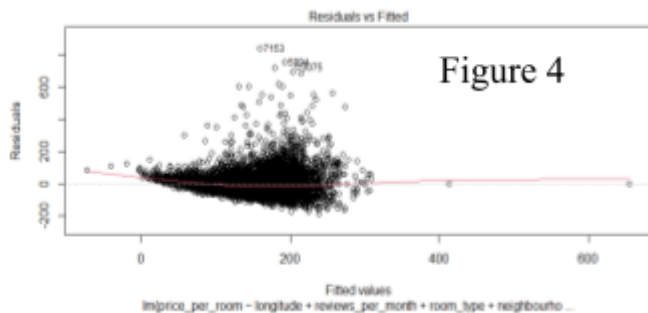
Figure 3 Price per room by Room Type



2. The initial model following preliminary examinations consisted of 4 predictors: room_type, neighbourhood_cleansed, longitude and reviews_per_month with an adjusted R^2 of 0.2439. This model was then expanded upon using stepwise selection on a curated list of additional predictors. This list was constructed by expanding on relationships identified in the exploratory analysis. Notably, it was chosen to

only include review_scores_rating from the predictors consisting of review_scores in order to preserve parsimony as they were found to share high correlations with each other.

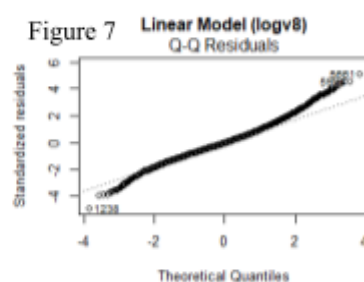
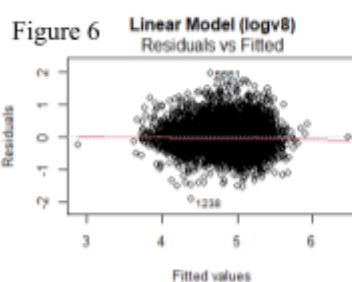
The resulting model incorporated 10 predictors and had an adjusted R^2 of 0.3001. This is a relatively small increase in R^2 given the number of predictors tripled, indicating that there may be fundamental modelling issues which are limiting model fit. By observing the residuals vs fitted curve of this model (Figure 4), the trend curve appears sloping. Additionally, the skew of price per room was found to be 2.316, indicating strong right-skew. This caused asymmetric residuals that ranged from -200 to 800, evidence of extreme positive outliers and heteroscedasticity. Thus the assumption of constant error variance and the stronger assumption of normally distributed errors are violated.



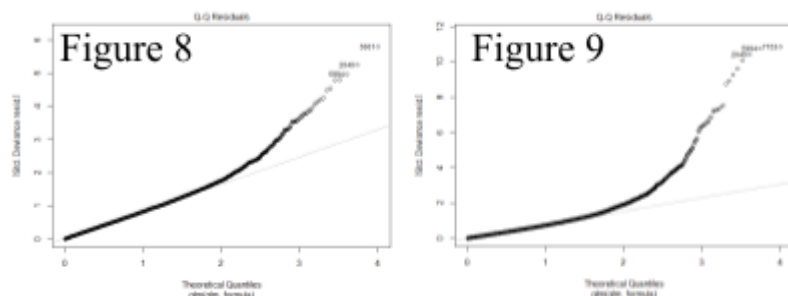
Thus, it was decided to transform Y to the log of price per room. This resulted in a model with an adjusted R^2 of 0.4363, significantly higher and notably resolving most of the heteroscedasticity (Figure 5). Proceeding from this model, adjustments were made to the additions from the stepwise process through the removal of longitude and latitude due to overlapping explanatory information similar to those in the initial model. Property type was also found to be a highly inefficient predictor, increasing the adjusted R^2 from 0.4225 to 0.4363 with 50 extra coefficients. The predictor property type consisted of 50 unique categories, of which the majority were found to be insignificant by the t-test. It was therefore removed and replaced by a binary classification for whether the property type was a houseboat, the most significant property type with a t-test value of 0.00699. Beyond existing predictors, new predictors amenity count and guests per bedroom were created to capture extra information on the quality of a listing through the unique amenities provided and to try to resolve collinearity between the accommodates and bedrooms variables respectively.

The final model was primarily selected through comparison of AIC and BIC to penalise complexity, and adjusted R^2 to measure explanatory power. It consisted of 11 predictors for the log of price per room. Thus, by undoing the log transformation (see appendix), a multiplicative rather than additive relationship is created between the coefficients and price per room such that changes in the coefficients cause an $\exp(\text{predictor}) - 1$ percent change in price. This model yielded an adjusted R^2 of 0.4513, an AIC of 7466.69 and a BIC of 7830.02. Moreover, the majority of predictors are highly significant with p-values below 0.001 from a t-test, although some categorical predictor coefficients were found to be insignificant, indicating that the model may be close to the best possible model with the constraints placed.

This model substantially improves upon the baseline by largely mitigating the violations to the assumptions made in linear modelling and is demonstrated by the diagnostic plots below. The residuals vs fitted plot (Figure 6) shows a random scattering of points evenly distributed above and below 0 whilst the maximum residual has decreased from 800 on the price scale to around 2 on the log scale. This provides evidence that heteroscedasticity has been mostly resolved. Additionally, the QQ-plot (Figure 7) demonstrates a reasonable fit along the central quantiles, but deviates from the tails. Although residual normality is imperfect, violating the strong assumption, the large observation count permits the application of the central limit theorem. This supports the validity of coefficient estimates made by least squares estimation and the associated hypothesis tests in the linear regression calculations. Furthermore, the accommodates and bedrooms predictors exhibited strong correlation with VIF values of approximately 12. Although their significance under the t-test justifies their inclusion, coefficient estimates may be unstable and should be treated cautiously when individually evaluated.



3. Based on the analysis done in part 2, an initial set of systematic predictors was created for the generalised linear model (GLM) using the same equation as the final linear regression model to facilitate analysis. Beyond the systematic predictors, GLMs require the specification of a distribution family (Stochastic component) and a link function.



Gamma model (Figure 8). Additionally, the AIC of the Gaussian model exceeded the Gamma model by over 4500, a clear indication of a worse fit.

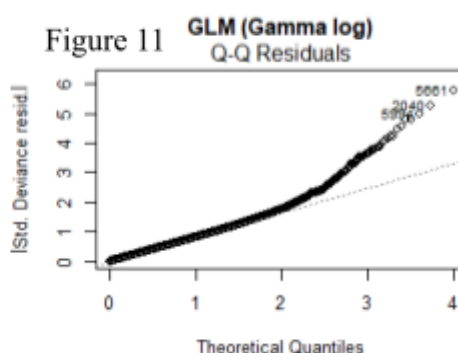
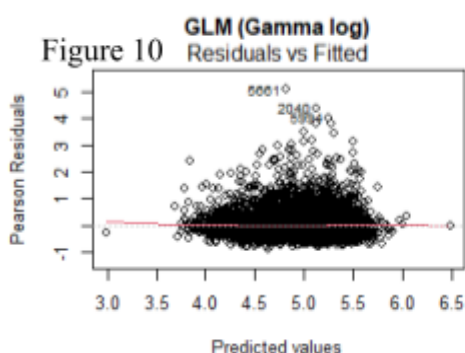
As price per room is a continuous, positive response, the appropriate stochastic distribution families are either Gaussian or Gamma. Recalling the high skew in the price per room dataset, it follows that a normal Gaussian model would be an unsuitable fit for this data and this is confirmed by the QQ-plots of the GLM residuals where the Gaussian model (Figure 9) deviates substantially more at the upper tail regions than a

The Gamma model has 3 common links, (Log, Inverse, Identity) so these models were compared to determine the best fit. However, when creating the identity link GLM, errors occurred due to the creation of negative values which produced NaNs, causing optimisation failures that prevented the calculation of viable coefficients. These errors persisted after supplying starting coefficients from a linear model, indicating a fundamental mismatch between the identity link and the dataset. While GLMs do not produce an R^2 output due to their calculation by Maximum Likelihood Estimation rather than Ordinary Least Squares, an R^2 analogue (pseudo- R^2) can be calculated using $1 - (\text{residual deviance} / \text{null deviance})$. This was used along with AIC and residual deviance to compare the Log and Inverse models with the results shown to the right. These results support a log link, demonstrating a better data fit through lower residual deviance and higher pseudo- R^2 , whilst also yielding lower AIC.

Thus, the GLM will be Gamma with a log link function.

	AIC	Pseudo_R_squared	Residual_deviance
log	86351.49	0.4272	0.1560
inverse	86431.23	0.4216	0.1575

After a brief stepwise selection process, none of the 11 predictors imported from the LR-model were dropped, strengthening the justification of the systemic predictor set used to create the final GLM model. Due to the log link



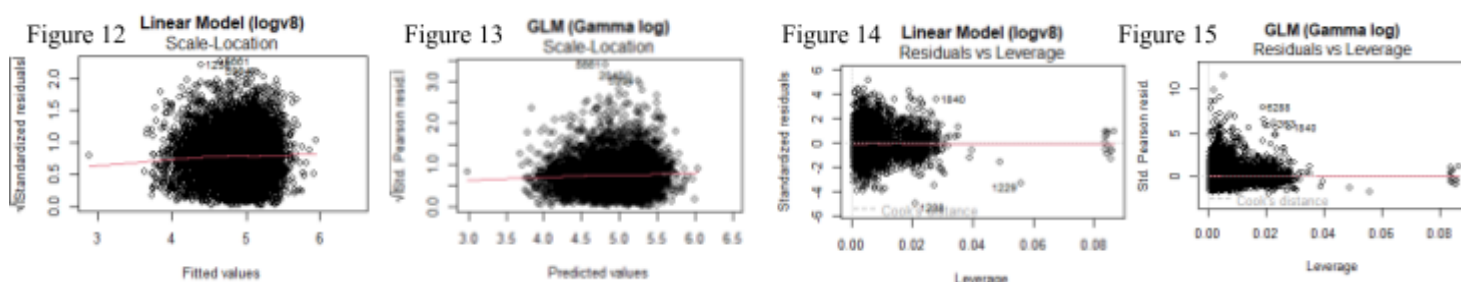
transformation, a multiplicative relationship is created between predictors and price per room, interpreted as percentage changes using $\exp(\text{predictor}) - 1$, consistent with the log-transformed LR-model. The final model's predictors continued to be significant with the majority of p-values produced from a t-test less than 0.001.

The Pearson residuals vs fitted plot (Figure 10) shows the majority of residuals lying between -1 and 1, adequately capturing the mean structure of the data where the residuals are mostly scattered around zero with a constant variance. These diagnostics suggest that the Gamma log-link model is appropriate. While there are large positive outliers such as observations 5661 and 2040 and a somewhat uneven distribution of observations with positive residuals, these do not indicate a systemic underprediction along the upper tail as the data does not produce an upwards trending line of fit. Instead, this reflects the extreme luxury outliers which exist in the dataset and affect the skew, creating isolated predictions with large positive residuals. This is consistent with the upper-tail deviations in the residuals QQ-plot (above right) confirming that the model is limited by its inability to capture extremely expensive listings successfully.

Q4. Linear models, such as the model produced within part 2, under the strong assumptions, are special cases of the GLM framework and so the GLM model produced within part 3 should be at minimum as effective. However, as the LR-model predicts $\log(\text{price per room})$ whilst the GLM predicts price per room, the AIC figures cannot be directly compared. Therefore, the LR-model is backtransformed to predict price per room and the models' predictive accuracy

is compared on the full dataset using root mean squared error (RMSE). RMSE is used because the pseudo- R^2 of the GLM and the adjusted R^2 of the LR-model are derived from different underlying quantities (Deviance and Variance) and are not directly comparable.

This produced the results shown on the left, demonstrating that the GLM produces marginally less error at \$71.2 to the \$72.6 of the LR-model. Given the median prices per room was around \$150, a \$1.5 difference in price prediction accuracy indicates that the practical improvement in predictive accuracy by the GLM was modest. This statistical benefit arises from the GLM's better handling of the mean variance relationship in price data, demonstrated best in the scale-location plots below.



The LR-model (Figure 12) has a downwards concave trendline that reflects the persistence of residual heteroscedasticity, not fully resolved by the log-transformation. This indicates that the variance of error terms is not constant and independent of the mean, violating the assumption of homoscedasticity. In contrast, the GLM (Figure 13) produces a substantially flatter trendline due to its ability to model variance as a function of mean through the Gamma distribution family. Thus, the GLM's improvement in RMSE arises by successfully reducing this heteroscedasticity error, more accurately modelling the mean-variance relationship in the price data.

The primary remaining flaw in the Gamma model is the handling of extreme outliers. The QQ-plots (Figures 7 & 11) show that both models sufficiently handled central quantiles but misrepresented upper tail quantiles. While the LR-model also experienced lower tail deviances, the GLM exhibited more pronounced upper tail deviation. This is further evidenced in the residual leverage plots where the GLM flags observations with standardised Pearson residuals that reached 12 (Figure 15), more than double the maximum seen in the LR-model (Figure 14). The key observation in the GLM is the recurrence of specific outlier observations (5661, 2040, 5994), presenting strong evidence that these luxury listings followed a unique pricing relationship driven by characteristics that were unobserved or not included in this model. This flaw is broadly consistent with the findings through RMSE, confirming the GLM's improvement in predictive ability to be partially offset by a greater sensitivity to anomalous high-price observations.

Q5. From the analysis of price per room, the largest influence on price per room was location, where coastal suburbs generally demanded premiums over their western counterparts. Neighbourhood coefficients indicated premiums of up to 65% in the most expensive suburb of Pittwater and a 37% discount in Fairfield reflecting desirability of locations as a clear price inflator. Room-type was another important determinant, as private rooms saw a ~30% discount when compared to entire home/apt listings. For investors, this provides clear incentives to rent out entire units as opposed to individual rooms for greater profits. The predictor guests per bedroom was highly significant, with each additional guest per bedroom corresponding to a 21% price increase. This indicates to investors that rental properties may benefit from being converted into open-plan properties rather than maintaining bedrooms. Various listing quality indicators increased prices, such as review score ratings, where 1-point improvements resulted in 24% premiums. Additionally, value can be found by increasing the count of luxuries in listings, as bathroom counts corresponded to 14.9% premiums whilst each extra amenity resulted in a 0.16% price increase. The negative coefficient for reviews per month indicated an association between lower prices and higher review counts. This suggests that for premium listings, investors should target fewer, high-value bookings rather than maximising occupancy. Perhaps the most important information for investors may come from the Superhost status, a coefficient not included in the final model. Superhost status was found to be statistically insignificant, suggesting that investors should prioritise increasing the intrinsic listing quality over platform specific marketing designations.

Appendix:

Model Summary:

Q2. Linear Regression Model

```
> summary(logv8)
```

Call:

```
lm(formula = log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +  
    review_scores_rating + accommodates + bathrooms + bedrooms +  
    is_houseboat + host_response_time + guests_per_bedroom +  
    amenity_count, data = Airbnb_data)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.88024	-0.24278	-0.02633	0.21842	1.96586

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	3.2206094	0.0944961	34.082	< 2e-16	***
reviews_per_month	-0.0599062	0.0031715	-18.889	< 2e-16	***
room_typePrivate room	-0.3675874	0.0132456	-27.752	< 2e-16	***
neighbourhood_cleansedAuburn	0.1057553	0.0563996	1.875	0.060815	.
neighbourhood_cleansedBankstown	-0.1332469	0.0633451	-2.104	0.035453	*
neighbourhood_cleansedBlacktown	-0.2030715	0.0612153	-3.317	0.000913	***
neighbourhood_cleansedBotany Bay	0.1065356	0.0613787	1.736	0.082654	.
neighbourhood_cleansedBurwood	0.0498630	0.0729150	0.684	0.494090	
neighbourhood_cleansedCamden	-0.0577889	0.0734432	-0.787	0.431392	
neighbourhood_cleansedCampbelltown	-0.1599603	0.0766507	-2.087	0.036931	*
neighbourhood_cleansedCanada Bay	0.1752781	0.0621701	2.819	0.004824	**
neighbourhood_cleansedCanterbury	-0.1356135	0.0684766	-1.980	0.047689	*
neighbourhood_cleansedCity Of Kogarah	-0.0663711	0.0770632	-0.861	0.389123	
neighbourhood_cleansedFairfield	-0.3651613	0.0745074	-4.901	9.72e-07	***
neighbourhood_cleansedHolroyd	-0.1943715	0.0786585	-2.471	0.013491	*
neighbourhood_cleansedHornsby	0.1147353	0.0583159	1.967	0.049163	*
neighbourhood_cleansedHunters Hill	0.1800695	0.1222223	1.473	0.140711	
neighbourhood_cleansedHurstville	-0.0727907	0.0810865	-0.898	0.369377	
neighbourhood_cleansedKu-Ring-Gai	0.0233429	0.0615889	0.379	0.704689	
neighbourhood_cleansedLane Cove	0.2783457	0.0756245	3.681	0.000234	***
neighbourhood_cleansedLeichhardt	0.2794351	0.0578926	4.827	1.41e-06	***
neighbourhood_cleansedLiverpool	-0.2691990	0.0716378	-3.758	0.000173	***
neighbourhood_cleansedManly	0.5512057	0.0545744	10.100	< 2e-16	***
neighbourhood_cleansedMarrickville	0.2243165	0.0565738	3.965	7.40e-05	***
neighbourhood_cleansedMosman	0.4199367	0.0644247	6.518	7.55e-11	***
neighbourhood_cleansedNorth Sydney	0.4515372	0.0548076	8.239	< 2e-16	***
neighbourhood_cleansedParramatta	-0.0096260	0.0599633	-0.161	0.872466	
neighbourhood_cleansedPenrith	-0.0664068	0.0655877	-1.012	0.311335	
neighbourhood_cleansedPittwater	0.6528718	0.0544279	11.995	< 2e-16	***
neighbourhood_cleansedRandwick	0.3777075	0.0538690	7.012	2.55e-12	***
neighbourhood_cleansedRockdale	0.0190051	0.0585140	0.325	0.745344	
neighbourhood_cleansedRyde	0.1023279	0.0587107	1.743	0.081387	.
neighbourhood_cleansedStrathfield	-0.0533961	0.0713151	-0.749	0.454039	
neighbourhood_cleansedSutherland Shire	0.2909603	0.0560554	5.191	2.15e-07	***
neighbourhood_cleansedSydney	0.4209992	0.0514352	8.185	3.15e-16	***
neighbourhood_cleansedThe Hills Shire	0.0341841	0.0606645	0.563	0.573114	
neighbourhood_cleansedWarringah	0.3584716	0.0546317	6.562	5.66e-11	***
neighbourhood_cleansedWaverley	0.4974760	0.0526981	9.440	< 2e-16	***
neighbourhood_cleansedWilloughby	0.0924932	0.0610070	1.516	0.129532	
neighbourhood_cleansedWoollahra	0.5042178	0.0571526	8.822	< 2e-16	***
review_scores_rating	0.2136838	0.0139432	15.325	< 2e-16	***

accommodates	-0.0471395	0.0063570	-7.415	1.34e-13	***
bathrooms	0.1386178	0.0090907	15.248	< 2e-16	***
bedrooms	-0.0492415	0.0126201	-3.902	9.63e-05	***
is_houseboat	1.3833000	0.3854301	3.589	0.000334	***
host_response_timenever responds	0.1515519	0.0452355	3.350	0.000811	***
host_response_timewithin a day	0.1971729	0.0473218	4.167	3.12e-05	***
host_response_timewithin a few hours	0.1368755	0.0455831	3.003	0.002684	**
host_response_timewithin an hour	0.0923208	0.0441582	2.091	0.036588	*
guests_per_bedroom	0.1894392	0.0103362	18.328	< 2e-16	***
amenity_count	0.0016301	0.0003202	5.091	3.65e-07	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3846 on 7949 degrees of freedom
Multiple R-squared: 0.4548, Adjusted R-squared: 0.4513
F-statistic: 132.6 on 50 and 7949 DF, p-value: < 2.2e-16

Q3. Generalised linear model

```
> summary(glm_final)
```

Call:

```
glm(formula = glm_formula, family = Gamma(link = "log"), data = Airbnb_data)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	3.3128517	0.1087712	30.457	< 2e-16	***
reviews_per_month	-0.0627940	0.0036506	-17.201	< 2e-16	***
room_typePrivate room	-0.3403304	0.0152465	-22.322	< 2e-16	***
neighbourhood_cleansedAuburn	0.1126707	0.0649196	1.736	0.082684	.
neighbourhood_cleansedBankstown	-0.1362420	0.0729144	-1.869	0.061726	.
neighbourhood_cleansedBlacktown	-0.2550927	0.0704628	-3.620	0.000296	***
neighbourhood_cleansedBotany Bay	0.0857260	0.0706509	1.213	0.225023	
neighbourhood_cleansedBurwood	0.0897801	0.0839300	1.070	0.284786	
neighbourhood_cleansedCamden	-0.0522885	0.0845380	-0.619	0.536250	
neighbourhood_cleansedCampbelltown	-0.1522714	0.0882300	-1.726	0.084414	.
neighbourhood_cleansedCanada Bay	0.1538650	0.0715619	2.150	0.031578	*
neighbourhood_cleansedCanterbury	-0.1714475	0.0788211	-2.175	0.029648	*
neighbourhood_cleansedCity Of Kogarah	-0.0748239	0.0887048	-0.844	0.398966	
neighbourhood_cleansedFairfield	-0.4297414	0.0857629	-5.011	5.54e-07	***
neighbourhood_cleansedHolroyd	-0.2179167	0.0905411	-2.407	0.016115	*
neighbourhood_cleansedHornsby	0.1332498	0.0671254	1.985	0.047169	*
neighbourhood_cleansedHunters Hill	0.1448407	0.1406859	1.030	0.303261	
neighbourhood_cleansedHurstville	-0.1205902	0.0933359	-1.292	0.196394	
neighbourhood_cleansedKu-Ring-Gai	-0.0127734	0.0708929	-0.180	0.857017	
neighbourhood_cleansedLane Cove	0.2876142	0.0870487	3.304	0.000957	***
neighbourhood_cleansedLeichhardt	0.2502056	0.0666381	3.755	0.000175	***
neighbourhood_cleansedLiverpool	-0.3158920	0.0824598	-3.831	0.000129	***
neighbourhood_cleansedManly	0.5415539	0.0628188	8.621	< 2e-16	***
neighbourhood_cleansedMarrickville	0.2015915	0.0651202	3.096	0.001970	**
neighbourhood_cleansedMosman	0.4236919	0.0741570	5.713	1.15e-08	***
neighbourhood_cleansedNorth Sydney	0.4591200	0.0630871	7.278	3.72e-13	***
neighbourhood_cleansedParramatta	-0.0234763	0.0690217	-0.340	0.733768	
neighbourhood_cleansedPenrith	-0.0654965	0.0754957	-0.868	0.385666	
neighbourhood_cleansedPittwater	0.6644854	0.0626501	10.606	< 2e-16	***
neighbourhood_cleansedRandwick	0.3748951	0.0620067	6.046	1.55e-09	***
neighbourhood_cleansedRockdale	-0.0124327	0.0673534	-0.185	0.853557	
neighbourhood_cleansedRyde	0.0915687	0.0675799	1.355	0.175466	
neighbourhood_cleansedStrathfield	-0.0867947	0.0820884	-1.057	0.290392	

neighbourhood_cleansedSutherland Shire	0.2959167	0.0645235	4.586	4.58e-06	***
neighbourhood_cleansedSydney	0.4158456	0.0592054	7.024	2.34e-12	***
neighbourhood_cleansedThe Hills Shire	0.0977155	0.0698288	1.399	0.161744	
neighbourhood_cleansedWarringah	0.3695292	0.0628847	5.876	4.36e-09	***
neighbourhood_cleansedWaverley	0.5029868	0.0606590	8.292	< 2e-16	***
neighbourhood_cleansedWilloughby	0.0720931	0.0702231	1.027	0.304626	
neighbourhood_cleansedWoollahra	0.4881560	0.0657864	7.420	1.29e-13	***
review_scores_rating	0.2081158	0.0160496	12.967	< 2e-16	***
accommodates	-0.0518334	0.0073174	-7.084	1.52e-12	***
bathrooms	0.1520728	0.0104640	14.533	< 2e-16	***
bedrooms	-0.0315242	0.0145265	-2.170	0.030028	*
is_houseboat	1.3286241	0.4436554	2.995	0.002755	**
host_response_timenever responds	0.1494430	0.0520690	2.870	0.004114	**
host_response_timewithin a day	0.1587271	0.0544705	2.914	0.003578	**
host_response_timewithin a few hours	0.1108061	0.0524691	2.112	0.034732	*
host_response_timewithin an hour	0.0610344	0.0508290	1.201	0.229872	
guests_per_bedroom	0.2004121	0.0118976	16.845	< 2e-16	***
amenity_count	0.0013178	0.0003686	3.575	0.000352	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 0.1959748)

Null deviance: 2164.3 on 7999 degrees of freedom
 Residual deviance: 1239.8 on 7949 degrees of freedom
 AIC: 86351

Number of Fisher Scoring iterations: 7

Log transformation (Q2/3)

$$\ln(Y) = \beta_0 + \beta_1 X + \varepsilon, \quad (1)$$

$$Y = \exp(\beta_0 + \beta_1 X + \varepsilon), \quad (2)$$

$$Y = e^{\beta_0} e^{\beta_1 X} e^{\varepsilon}, \quad (3)$$

$$Y(X+1) = e^{\beta_0} e^{\beta_1(X+1)} e^{\varepsilon}, \quad (4)$$

$$= e^{\beta_0} e^{\beta_1 X} e^{\beta_1} e^{\varepsilon}, \quad (5)$$

$$= Y(X) e^{\beta_1}, \quad (6)$$

$$\frac{Y(X+1) - Y(X)}{Y(X)} = \frac{Y(X)e^{\beta_1} - Y(X)}{Y(X)}, \quad (7)$$

$$= e^{\beta_1} - 1. \quad (8)$$

Bibliography

(No date a) *10.7 - detecting multicollinearity using variance inflation factors | Stat 462*. Available at: <https://online.stat.psu.edu/stat462/node/180/> (Accessed: 03 July 2026).

(No date b) *Understand variance inflation factor (VIF) in regression analysis*. Available at: <https://www.investopedia.com/terms/v/variance-inflation-factor.asp> (Accessed: 03 July 2026).

(No date c) *Stepwise regression essentials in R - articles - STHDA*. Available at: <https://www.sthda.com/english/articles/37-model-selection-essentials-in-r/154-stepwise-regression-essentials-in-r/> (Accessed: 03 July 2026).

(No date d) *Most basic lollipop plot – the R graph gallery*. Available at: <https://r-graph-gallery.com/300-basic-lollipop-plot.html> (Accessed: 03 July 2026).

Generative AI usage

Question 1:

- Assisted in generation of graphs – handling neighbourhood_cleansed and ordering for the creation of the lollipop chart
- Helped debug and clean categorical data – Introduced idea that blank observations in superhost status could provide information, explaining why it was not a binary category. Helped recategorise blank observations as “not_evaluated”

Question 2:

- Provided explanation of the stepAIC function used in stepwise modelling process
- Helped produce the curated list of predictors used for stepwise modelling
- Guided me in understanding why a log transformation would assist the model fit
- Helped debug modelling issues from the introduction of guests_per_bedroom when observations fitted decreased – see logv6fixed

Question 3:

- Guided me on which stochastic and link functions were relevant to test
- Helped interpret plots and information to determine which stochastic and link functions were the best fit

Question 4:

- Provided relevant comparison tools to compare GLM and LR – RMSE; explained and guided its calculation process through generating fit predictions
- Helped interpret plots

Misc usage:

- Explained concepts that were unfamiliar
- Debugged code
- Provided general feedback and corrections to report terminology and claims
- Provided grammatical corrections

R code:

```
Airbnb_data <- read.csv(file="C:/Users/simon/Documents/3142_ass1/airbnb-Sydney.csv", header=TRUE, na.strings =
"N/A")

library(dplyr)
library(ggplot2)
library(MASS)
library(moments)
library(car)
library(stringr)

#Q1
#numeric predictors
numeric_data <- Airbnb_data[, sapply(Airbnb_data, is.numeric)]
numeric_data <- na.omit(numeric_data)

#calculate price correlations
price_cor <- cor(numeric_data$price_per_room, numeric_data)

#remove cor with x (counting column) and price_per_room
price_cor <- price_cor[, -c(1,33)]

#order correlations in ascending order
sorted_price_cor <- sort(price_cor, decreasing = F)

#increase plot limits
par(mfrow = c(1,1))
par(mar = c(13, 4, 4, 2))

#plot Correlations with price per room
barplot(sorted_price_cor,
        las = 2,
        col = "tomato",
        main = "Correlations with price_per_room",
        ylab = "Correlation")

#categorical predictors
#room_type

#create box plots of price per room based on room type
#Calculate medians of price per room based on room type
tapply(Airbnb_data$price_per_room,
        Airbnb_data$room_type,
        median)

#plot
ggplot(Airbnb_data, aes(x = reorder(room_type, price_per_room, median),
                        y = price_per_room, fill = room_type)) +
#remove outliers from graph (still used in calculations)
geom_boxplot(outlier.shape = NA) +
coord_flip() +
labs(title = "Price per room by Room Type",
```

```
x = "Room Type",
y = "Price per Room") +
theme_classic()
```

```
#Create lollipop chart of median price per rooms by neighbourhood
#Group data by neighbourhoods and arrange in order by median price
```

```
Airbnb_data %>%
  group_by(neighbourhood_cleansed) %>%
  summarise(median_price = median(price_per_room)) %>%
  arrange(median_price) %>%
  mutate(neighbourhood_cleansed = factor(
    neighbourhood_cleansed,
    levels = neighbourhood_cleansed
  )) %>%
```

```
#Create lollipop chart
ggplot(aes(x = neighbourhood_cleansed, y = median_price)) +
  geom_segment(aes(xend = neighbourhood_cleansed, y = 0, yend = median_price)) +
  geom_point() +
  coord_flip() +
  labs(title = "Median Price per Room by Neighbourhood",
    x = "Neighbourhood",
    y = "Median Price per Room") +
  theme_classic()
```

```
#Calculate Correlations between Review Score ratings
cor(Airbnb_data[, c("review_scores_rating", "review_scores_location",
  "review_scores_cleanliness", "review_scores_accuracy",
  "review_scores_checkin", "review_scores_communication",
  "review_scores_value")], use = "complete.obs")
```

```
#Additional Data manipulation for models (cleaning up data)
```

```
#Host is superhost was not a binary variable
table(Airbnb_data$host_is_superhost, useNA = "always")
```

```
#Blank returns are not NAs but indicate hosts who have not been evaluated to superhosts
#Thus T --> High quality host, F --> Lower quality hosts, NA --> new/inactive host
#Reclassify data to represent this fact
```

```
Airbnb_data$host_is_superhost <- as.character(Airbnb_data$host_is_superhost)
Airbnb_data$host_is_superhost[Airbnb_data$host_is_superhost == ""] <- "not_evaluated"
Airbnb_data$host_is_superhost <- as.factor(Airbnb_data$host_is_superhost)
```

```
#plot host_is_superhost vs price_per_room
```

```
ggplot(Airbnb_data, aes(x = reorder(host_is_superhost, price_per_room, median),
  y = price_per_room, fill = host_is_superhost)) +
  geom_boxplot(outlier.shape = NA) +
  coord_flip(ylim = c(0,400)) +
  labs(title = "Price per room by Host Status",
```

```
x = "Host Status",
y = "Price per Room") +
theme_classic()
```

```
#host_response_time
table(Airbnb_data$host_response_time, useNA = "always")
unique(Airbnb_data$host_response_time)
```

```
#Na responses in host_response_time likely represents when listings don't respond
#rather than lack of data
```

```
#Make NA responses into "never_responds"
Airbnb_data$host_response_time <- as.character(Airbnb_data$host_response_time)
Airbnb_data$host_response_time[is.na(Airbnb_data$host_response_time)] <- "never_responds"
Airbnb_data$host_response_time <- as.factor(Airbnb_data$host_response_time)

table(Airbnb_data$host_response_time, useNA = "always")
```

```
#Q2
#initial model using highest correlation predictors
```

```
#starting with all 4
#drop longitude -- likely has overlap with neighbourhood cleansed
#drop longitude bc R^2 is smaller when longitude is dropped compared to when neighbourhood is dropped
```

```
lrv1 <- lm(formula = price_per_room ~ longitude + reviews_per_month +
           room_type + neighbourhood_cleansed, data = Airbnb_data)
summary(lrv1)
AIC(lrv1)
```

```
#Model without longitude
lrv2 <- lm(formula = price_per_room ~ reviews_per_month +
           room_type + neighbourhood_cleansed, data = Airbnb_data)
summary(lrv2)
AIC(lrv2)
```

```
#Longitude and neighbourhood had overlapping explanatory power:
#Longitude removed due to lower R^2
```

```
#Curated list of predictors
scope_formula <- list(
  lower = ~ 1,
  upper = ~ reviews_per_month +
    room_type +
    neighbourhood_cleansed +
    property_type +
    host_is_superhost +
    review_scores_rating +
    accommodates + bathrooms +
    longitude +
    latitude
)
```

#Produce Step model based on predictors

```
step.model <- stepAIC(lrv1, direction = "both", scope = scope_formula, trace = F)
summary(step.model)
```

```
step.model <- lm(formula = price_per_room ~ longitude + reviews_per_month +
  room_type + neighbourhood_cleansed + property_type + review_scores_rating +
  accommodates + bathrooms + latitude + host_is_superhost,
  data = Airbnb_data)
plot(step.model)
```

#residual vs fitted -- spread of residuals fan out from 0-300 then narrow again -- models errors are not constant
#evidence of heteroscedasticity (violates assumption) -- should take the log and replot

#Change 1: Removed property type -- Introduces many coefficients for limited improvement

#Adj R²: 0.2665

```
modelv1 <- lm(price_per_room ~ longitude + reviews_per_month +
  room_type + neighbourhood_cleansed + review_scores_rating +
  accommodates + bathrooms + latitude + host_is_superhost,
  data = Airbnb_data)
summary(modelv1)
```

#Change 2: Removed longitude and latitude - Neighbourhood_cleansed better represents this information

#Added bedrooms

#Adj R²: 0.2815

```
modelv2 <- lm(price_per_room ~ reviews_per_month +
  room_type + neighbourhood_cleansed + review_scores_rating +
  accommodates + bathrooms + bedrooms + host_is_superhost,
  data = Airbnb_data)
summary(modelv2)
```

#Change 3: Large Skew (2.31) and Kurtosis (13.92)

```
skewness(Airbnb_data$price_per_room)
```

```
kurtosis(Airbnb_data$price_per_room)
```

#residual vs fitted -- spread of residuals fan out from 0-300 then narrow again -- models errors are not constant

#evidence of heteroscedasticity (violates assumption)

#Model Log price per room rather than price per room

#Create log price per room variable

```
Airbnb_data$log_price <- log(Airbnb_data$price_per_room)
```

#Initial log model to compare with step model

#Adj R²: 0.4363

```
log_step_model <- lm(log_price ~ longitude + reviews_per_month +
  room_type + neighbourhood_cleansed + property_type + review_scores_rating +
  accommodates + bathrooms + latitude + host_is_superhost,
  data = Airbnb_data)
summary(log_step_model)
plot(log_step_model)
```

#Change 4: Apply changes made to step model to the initial log model

#Change 5: Remove property type

```

#Adj R^2: 0.4225
logv2 <- lm(log_price ~ reviews_per_month +
            room_type + neighbourhood_cleansed + review_scores_rating +
            accommodates + bathrooms + host_is_superhost + bedrooms,
            data = Airbnb_data)

summary(logv2)

#Property Type analysis

#only a few were significant
#property_typeHouseboat 0.00699 **
#property_typePrivate room in chalet 0.01033 *
#property_typePrivate room in villa 0.03816 *

#Majority insignificant -- 1.7% increase in Rsquared -- 55 Extra coefficients

#Create a binary predictor for houseboat, the most significant property type
Airbnb_data$is_houseboat <- ifelse(Airbnb_data$property_type == "Houseboat", 1, 0)

#Change 6: is_houseboat used as replacement for property type
#logv3 and logv4 used to compare the influence of host_is_superhost
#Without host_is_superhost

#Adj R^2: 0.423
logv3 <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
            review_scores_rating + accommodates + bathrooms +
            bedrooms + is_houseboat,
            data = Airbnb_data)
summary(logv3)

#With host_is_superhost
#Adj R^2: 0.4233
logv4 <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
            review_scores_rating + accommodates + bathrooms +
            bedrooms + is_houseboat + host_is_superhost,
            data = Airbnb_data)
summary(logv4)

#comparison of models
AIC(logv3) # without host_is_superhost
AIC(logv4) # with host_is_superhost
BIC(logv3)
BIC(logv4)

#Change 7: host_is_superhost removed -- Found to be insignificant
#Change 8: add host_response_time
#Adj R^2: 0.4258
logv5 <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
            review_scores_rating + accommodates + bathrooms +
            bedrooms + is_houseboat + host_response_time,
            data = Airbnb_data)
summary(logv5)

```



```

cor(Airbnb_data$accommodates, Airbnb_data$bedrooms)
#accommodates and bedrooms have high correlation -- 0.87
#create guests per room -- accomodates/ bedrooms to replace

Airbnb_data$guests_per_bedroom <- ifelse(
  Airbnb_data$bedrooms > 0, Airbnb_data$accommodates/Airbnb_data$bedrooms,
  Airbnb_data$accommodates / 1
)

#Change 9: guests_per_bedroom introduced to replace accommodates and bedrooms

logv6 <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
  review_scores_rating + bathrooms +is_houseboat + host_response_time
  + guests_per_bedroom,
  data = Airbnb_data)
summary(logv6)

#Problems introduced by model logv6:
#Listings with 0 bedrooms create NaNs
#Problem resolved by setting to equal accommodates
#host_response_time includes large amounts of blank responses
#Problems resolved in additional data manipulations section

#Model with problems resolved

#Adj R^2: 0.4136
logv6fixed <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
  review_scores_rating + bathrooms +is_houseboat + host_response_time
  + guests_per_bedroom,
  data = Airbnb_data)
summary(logv6fixed)

#Change 10: Reintroduced bedrooms, accommodates
#Adj R^2: 0.4496
logv7 <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
  review_scores_rating + accommodates + bathrooms + bedrooms +
  is_houseboat + host_response_time + guests_per_bedroom,
  data = Airbnb_data)

summary(logv7)
vif(logv7)
#Both accommodates and bedrooms contribute to highly variability
#Still remain significant to model -- retain all 3 predictors

#Create amenity count predictor
Airbnb_data$amenity_count <- str_count(Airbnb_data$amenities, ",") + 1

#Change 11: Add amenity Count
#Adj R^2: 0.4513
logv8 <- lm(log_price ~ reviews_per_month + room_type + neighbourhood_cleansed +
  review_scores_rating + accommodates + bathrooms + bedrooms +

```

```

    is_houseboat + host_response_time + guests_per_bedroom + amenity_count,
    data = Airbnb_data)
summary(logv8)

#Final model Comparisons
AIC(logv3); AIC(logv5); AIC(logv6fixed); AIC(logv7); AIC(logv8)
BIC(logv3); BIC(logv5); BIC(logv6fixed); BIC(logv7); BIC(logv8)

#Final model
lm_final <- logv8

summary(lm_final)
AIC(lm_final)
BIC(lm_final)
vif(lm_final)

#Q3
#take final formula from linear regression model for initial analysis
glm_formula <- price_per_room ~ reviews_per_month + room_type + neighbourhood_cleansed +
  review_scores_rating + accommodates + bathrooms + bedrooms +
  is_houseboat + host_response_time + guests_per_bedroom + amenity_count

glmv1 <- glm(glm_formula, family = Gamma(link = "log"), data = Airbnb_data)
summary(glmv1)

#Choosing Stochastic function: Gaussian vs Gamma

#Create Gaussian model
glm_gaussian <- glm(glm_formula, family = gaussian(link = "log"), data = Airbnb_data)
glm_gamma <- glm(glm_formula, family = Gamma(link = "log"), data = Airbnb_data)

AIC(glm_gaussian) - AIC(glm_gamma)
#4584.529 > Gamma structure fits better

plot(glm_gaussian)
plot(glm_gamma)

#Choosing Link Function for Gamma: Log vs Inverse vs Identity

glmv1_log <- glm(glm_formula, family = Gamma(link = "log"), data = Airbnb_data)
glmv1_inverse <- glm(glm_formula, family = Gamma(link = "inverse"), data = Airbnb_data)

#Identity link fails to run - No valid set of coefficients
#Due to negatives being passed
#Supply starting values using linear model
lm_start <- lm(glm_formula, data = Airbnb_data)

glmv1_identity <- glm(glm_formula, family = Gamma(link = "identity"),
  start = coef(lm_start), data = Airbnb_data)
#still fails to run -- Evidence that the identity link is insufficient

#comparison will continue between log and inverse
AIC(glmv1_log); AIC(glmv1_inverse)

```

```

glm_log_family_links <- list(
  "log" = glmv1_log,
  "inverse" = glmv1_inverse
)

data.frame(
  #AIC
  AIC = sapply(glm_log_family_links, AIC),

  Pseudo_R_squared = sapply(glm_log_family_links, function(m)
    round(1 - m$deviance / m$null.deviance, 4)),

  Residual_deviance = sapply(glm_log_family_links, function(m)
    round(m$deviance / m$df.residual, 4))
)

```

#Procede with Log link function

#Perform stepwise selection to ensure predictors are significant

#Null and full scope

```

glm_null <- glm(price_per_room ~ 1,
  family = Gamma(link = "log"),
  data = Airbnb_data)

```

```

glm_full <- glm(glm_formula,
  family = Gamma(link = "log"),
  data = Airbnb_data)

```

#Stepwise on GLM

```

glm_step <- stepAIC(glm_full,
  scope = list(lower = glm_null, upper = glm_full),
  direction = "both",
  trace = FALSE)

```

```
summary(glm_step)
```

#retained all variables

#FINAL GLM model

```

glm_final <- glmv1_log
summary(glm_final)

```

#Q4

#display summary datas together

```

par(mfrow = c(2,2))
plot(logv8, main = "Linear Model (logv8)")

```

```

par(mfrow = c(2,2))
plot(glm_final, main = "GLM (Gamma log)")

```

```
#Compare overall fit by generating predictions using original data
```

```
#linear model requires transformation back from log
```

```
pred_lm <- exp(fitted(lm_final))
```

```
pred_glm <- fitted(glm_final)
```

```
#Compute RMSE
```

```
actual <- Airbnb_data$price_per_room
```

```
rmse_lm <- sqrt(mean((actual - pred_lm)^2, na.rm = TRUE))
```

```
rmse_glm <- sqrt(mean((actual - pred_glm)^2, na.rm = TRUE))
```

```
data.frame(
```

```
  RMSE = c(rmse_lm, rmse_glm),
```

```
  row.names = c("logv8 (LM)", "glm_final (GLM)")
```

```
)
```